

Sitzung 4: Generierte Forschungssoftware

Dr. Julian Dehne

Gesellschaft für Informatik e.V. (GI) · deRSE
julian.dehne@gi.de · ORCID 0000-0001-9265-9619

Einordnung

Nach dieser Einheit können Sie ...

- einordnen, wofür Sprachmodelle im Software Engineering bisher untersucht werden,
- begründen, warum generierter Code in der Forschung andere Fragen aufwirft als in der Produktentwicklung,
- Risiken generierter Forschungssoftware benennen und einer Qualitätsdimension zuordnen,
- einschätzen, welche dieser Risiken sich im Code selbst kenntlich machen und prüfen lassen.

- RSE und generierter Code: Begriffe und Abgrenzung
- Stand der Forschung: Sprachmodelle im Software Engineering
- Was sich für Forschungssoftware ändert
- Diskussion und Bezug zur Projektaufgabe

RSE und generierter Code

Was heißt “generierte Forschungssoftware”?

- **Generierter Code:** Quellcode, den ein Sprachmodell ganz oder teilweise erzeugt hat, von der Zeilenvervollständigung bis zum agentisch erstellten Projekt.
- **Forschungssoftware:** Software, die im Forschungsprozess oder für einen Forschungszweck entstanden ist (Gruenpeter et al. 2021).
- **Generierte Forschungssoftware:** Beides zugleich. Der Code trägt ein Forschungsergebnis, und ein Teil davon stammt nicht von der Person, die das Ergebnis verantwortet.

- Vervollständigung einzelner Zeilen im Editor
- Erzeugung ganzer Funktionen oder Skripte aus einer Beschreibung
- Überarbeitung bestehenden Codes (Refactoring, Übersetzung, Fehlersuche)
- Agenten, die selbstständig planen, ausführen und testen

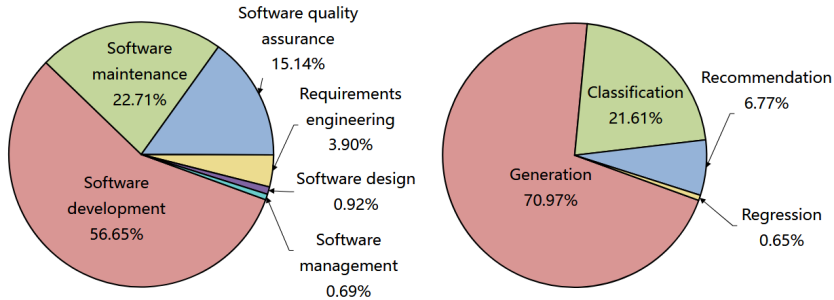
Je höher die Stufe, desto weniger vom Code hat ein Mensch gelesen.

Stand der Forschung

Systematische Literaturübersicht von Hou et al. (2024):

- 395 Studien aus den Jahren 2017 bis Anfang 2024
- Vier Fragen: Welche Modelle werden eingesetzt? Mit welchen Daten? Wie werden sie angepasst und bewertet? Für welche Aufgaben?
- Guter Einstieg in das Feld, weil die Übersicht die Aufgaben nach den Phasen des Software Engineering ordnet

Wofür Sprachmodelle untersucht werden



(a) Distribution of LLM usages in SE activities. (b) Problem classification based on collected studies.

Figure 1: Verteilung der untersuchten Einsätze nach SE-Aktivität (a) und nach Problemtyp (b) (Hou et al. 2024).

Die häufigsten Aufgaben je SE-Aktivität

SE-Aktivität	Studien	Häufigste Aufgaben (Anzahl Studien)
Requirements engineering	17	Anaphoric ambiguity treatment (4), Requirements classification (4), Requirement analysis and evaluation (2), Specification generation (2)
Software design	4	GUI retrieval, Rapid prototyping, Software specification synthesis, System design (je 1)
Software development	247	Code generation (118), Code completion (22), Code summarization (21), Code search und Code translation (je 12)
Software quality assurance	66	Vulnerability detection (18), Test generation (17), Bug localization (5), Verification (5)
Software maintenance	99	Program repair (35), Code clone detection (8), Code review (7), Debugging (4)
Software management	3	Effort estimation (2), Software tool configuration (1)

Zuordnungen nach Hou et al. (2024); eine Studie kann mehrere Aufgaben behandeln, daher 436 Zuordnungen bei 395 Studien.

Was die Verteilung zeigt

- Über die Hälfte der Studien betrifft die Softwareentwicklung (56,65 %), danach Wartung (22,71 %) und Qualitätssicherung (15,14 %).
- Anforderungen (3,90 %), Entwurf (0,92 %) und Management (0,69 %) kommen kaum vor.
- Rund 71 % der Aufgaben sind Generierungsaufgaben.

Die Forschung konzentriert sich auf das Erzeugen von Code. Die Phasen, in denen entschieden wird, *was* gebaut wird und *warum*, sind wenig untersucht.

Halluzinationen in generiertem Code



```
def tri(n):  
    """Everyone knows Fibonacci sequence, it was studied deeply by  
    mathematicians in the last couple centuries. However, what people  
    don't know is Tribonacci sequence.  
    Tribonacci sequence is defined by the recurrence:  
    tri(0) = 1  
    tri(n) = 1 + n / 2, if n is even.  
    tri(n) = tri(n - 1) + tri(n - 2) + tri(n + 1), if n is odd.  
    For example:  
    tri(2) = 1 + (2 / 2) = 2  
    tri(4) = 3  
    tri(3) = tri(2) + tri(1) + tri(4)  
           = 2 + 3 + 3 = 8  
    You are given a non-negative integer number n, you have to return  
    a list of the first n + 1 numbers of the Tribonacci sequence.  
    Examples:  
    tri(3) = [1, 3, 2, 8]  
    """
```

Code w/ Hallucination



```
def tri(n):  
    if n == 0:  
        return [1]  
    my_tri = [1, 1]  
    for i in range(2, n + 1):  
        if i % 2 == 0:  
            my_tri.append(i / 2 + 1)  
        else:  
            my_tri.append(my_tri[i - 1] + my_tri[i - 2] + (i + 3) / 2)  
    return my_tri
```

Code w/o Hallucination



```
def tri(n):  
    if n == 0:  
        return [1]  
    my_tri = [1, 3]  
    for i in range(2, n + 1):  
        if i % 2 == 0:  
            my_tri.append(i / 2 + 1)  
        else:  
            my_tri.append(my_tri[i - 1] + my_tri[i - 2] + my_tri[i + 1])  
    return my_tri
```

Fig. 1. Illustration of code with or without hallucination.

Liu et al. (2024): Was geht über funktionale Korrektheit hinaus schief?

- **Halluzination:** Die Ausgabe weicht von der Absicht ab, widerspricht sich selbst oder dem Weltwissen.
- Taxonomie aus generiertem Code: 3 Hauptkategorien, 12 Unterkategorien
- Verteilung über Modelle und Benchmarks, Ursachen und Folgen
- Gegenmaßnahmen allein über den Prompt

Im Beispiel verletzt der Code die Vorgabe $\text{tri}(1) = 3$ und läuft trotzdem.

Figure 2: Beispiel aus (Liu et al. 2024).

Taxonomie der Code-Halluzinationen

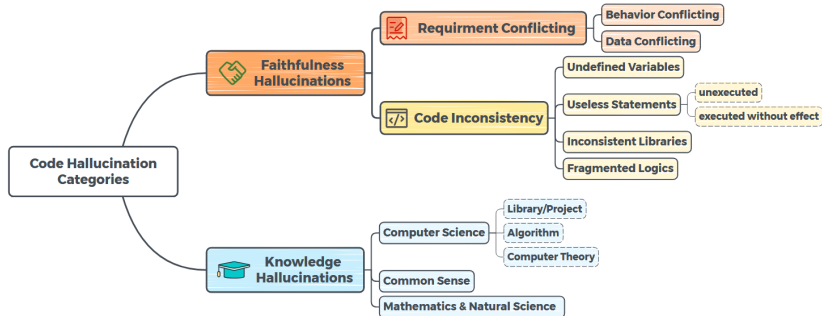


Figure 3: Kategorien von Code-Halluzinationen (Liu et al. 2024).

Vibe Coding: Praxis, Erleben und Folgen

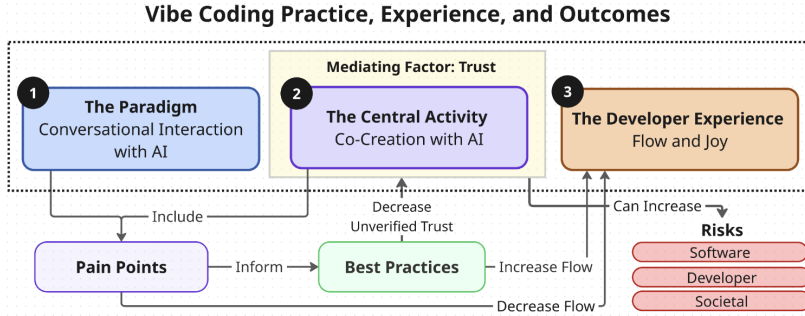


Figure 4: Vibe Coding: Praxis, Erleben und Folgen (Pimenova et al. 2025).

Generierter Code und Open-Source-Lizenzen

Xu et al. (2025) (LiCoEval, ICSE 2025): Hält generierter Code die Lizenzen seiner Vorlagen ein?

- **Herkunft:** Die Modelle sind mit lizenziertem Open-Source-Code trainiert und geben Teile davon samt Kommentaren fast wörtlich wieder.
- **Die Lizenz geht verloren:** Wiederverwendung ist erlaubt, aber an Bedingungen geknüpft, auch bei permissiven Lizenzen (z. B. Namensnennung und Änderungsvermerk bei Apache-2.0). Der generierte Code kommt ohne diese Angaben.
- **Befund:** 14 Modelle, 4.187 Python-Funktionen. Bei den drei stärksten Modellen sind 0,88 bis 2,01 % der Ausgaben einer Vorlage „auffallend ähnlich“ (*striking similarity*).
- **Nachfrage hilft kaum:** Die meisten Modelle nennen die Lizenz nicht korrekt. Bei Copyleft-Code liegt die Trefferquote bei allen Modellen bis auf eines bei null.
- **Nicht prüfbar:** Die Trainingsdaten sind meist nicht offengelegt.
- **Folge:** Das rechtliche Risiko liegt bei denen, die den Code verwenden.

Grenzen der Studie: nur Python, nur einzelne Funktionen, bewusst strenger Schwellenwert.

Zehn Regeln für KI-gestütztes Programmieren in der Wissenschaft

Bridgeford et al. (2025): *Ten Simple Rules for AI-Assisted Coding in Science*

1. Gather Domain Knowledge Before Implementation
2. Distinguish Problem Framing from Coding
3. Choose Appropriate AI Interaction Models
4. Start by Thinking Through a Potential Solution
5. Manage Context Strategically
6. Implement Test-Driven Development with AI
7. Leverage AI for Test Planning and Refinement
8. Monitor Progress and Know When to Restart
9. Critically Review Generated Code
10. Refine Code Incrementally with Focused Objectives

- Bridgeford, Eric W., Iain Campbell, Zijiao Chen, Zhicheng Lin, Harrison Ritz, Joachim Vandekerckhove, and Russell A. Poldrack. 2025. "Ten Simple Rules for AI-Assisted Coding in Science." <https://doi.org/10.48550/arXiv.2510.22254>.
- Gruenpeter, Morane, Daniel S. Katz, Anna-Lena Lamprecht, et al. 2021. "Defining Research Software: A Controversial Discussion." Zenodo. <https://doi.org/10.5281/zenodo.5504016>.
- Hou, Xinyi, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. "Large Language Models for Software Engineering: A Systematic Literature Review." *ACM Transactions on Software Engineering and Methodology* 33 (8): 1–79. <https://doi.org/10.1145/3695988>.
- Liu, Fang, Yang Liu, Lin Shi, Zhen Yang, Li Zhang, Xiaoli Lian, Zhongqi Li, and Yuchi Ma. 2024. "Beyond Functional Correctness: Exploring Hallucinations in LLM-Generated Code." <https://doi.org/10.48550/arXiv.2404.00971>.
- Pimenova, Veronica, Sarah Fakhoury, Christian Bird, Margaret-Anne Storey, and Madeline Endres. 2025. "Good Vibrations? A Qualitative Study of Co-Creation, Communication, Flow, and Trust in Vibe Coding." <https://doi.org/10.48550/arXiv.2509.12491>.

Xu, Weiwei, Kai Gao, Hao He, and Minghui Zhou. 2025. “LiCoEval: Evaluating LLMs on License Compliance in Code Generation.” In *Proceedings of the 47th International Conference on Software Engineering (ICSE 2025)*. <https://doi.org/10.48550/arXiv.2408.02487>.